

CAPSTONE PROJECT REPORT

AUTOMATION TESTING PRACTICE WEBSITE AUTOMATION FRAMEWORK

Submitted By

Bala Teja Surla

GitHubLink: <https://github.com/balateji/CapstonRepos>

Technologies Used

- Java
- Selenium WebDriver
- TestNG
- Maven
- Jenkins
- GitHub
- Extent Reports
- WebDriverManager

1. PROJECT OVERVIEW

The Automation Testing Practice Website Automation Framework was developed to automate the testing of web application functionalities using Selenium WebDriver and Java. The framework follows the Page Object Model (POM) design pattern to ensure maintainability, scalability, and code reusability.

The framework automates various user interactions such as form submission, alert handling, dropdown selection, file upload, web table validation, window handling, mouse actions, and broken link verification.

2. PROJECT OBJECTIVE

The primary objective of this project is to:

- Reduce manual testing effort.
- Improve test coverage.
- Increase execution efficiency.
- Detect application defects early.
- Enable Continuous Integration using Jenkins.

- Generate automated execution reports.
-

3. APPLICATION DETAILS

Application Name

Automation Testing Practice Website

Application URL

<https://testautomationpractice.blogspot.com/>

Browser Support

- Google Chrome
- Mozilla Firefox

Operating System

- Windows 10/11
-

4. TOOLS AND TECHNOLOGIES

Tool	Purpose
Java	Programming Language
Selenium WebDriver	Browser Automation
TestNG	Test Execution and Reporting
Maven	Build Management
Jenkins	Continuous Integration
GitHub	Source Code Management
Extent Reports	Reporting
WebDriverManager	Driver Management

5. FRAMEWORK DESIGN

The framework is implemented using the Page Object Model (POM).

Framework Components

- Base Test Class
- Page Classes
- Utility Classes
- Test Classes
- TestNG XML
- Jenkins Pipeline
- Extent Reports

Advantages

- Reusable code
 - Easy maintenance
 - Better scalability
 - Modular architecture
-

6. MODULES COVERED

Functional Modules

1. Form Submission
 2. Radio Button Validation
 3. Checkbox Validation
 4. Dropdown Handling
 5. Alert Handling
 6. Multiple Windows and Tabs
 7. Web Table Validation
 8. File Upload
 9. Dynamic Button
 10. Mouse Hover
 11. Double Click
 12. Drag and Drop
 13. Broken Link Validation
 14. Laptop Links Verification
 15. Pagination Web Table
-

7. TEST SCENARIOS

Form Validation

- Verify Name field.
- Verify Email field.

- Verify Phone Number field.
- Verify Address field.

Alert Handling

- Simple Alert
- Confirmation Alert
- Prompt Alert

Dropdown Handling

- Country Selection
- Color Selection

Window Handling

- Open New Window
- Switch Window
- Verify Content

Web Tables

- Retrieve Data
- Validate Rows
- Validate Columns

File Upload

- Upload Valid File
- Verify Upload Success

8. AUTOMATION EXECUTION FLOW

1. Launch Browser
 2. Navigate to Application
 3. Execute Test Cases
 4. Capture Results
 5. Generate Extent Report
 6. Publish Results
 7. Jenkins Build Execution
-

9. JENKINS INTEGRATION

The project is integrated with Jenkins Pipeline.

Pipeline Stages:

1. Checkout
2. Build
3. Compile
4. Test
5. Package
6. Report Generation

Jenkins automatically retrieves source code from GitHub, executes Maven commands, runs TestNG test suites, and publishes execution reports.

10. TEST EXECUTION SUMMARY

Metric	Count
Total Test Cases	16
Executed Test Cases	16
Passed Test Cases	16
Failed Test Cases	0
Blocked Test Cases	0

11. DEFECT ANALYSIS

High Severity Defects

- Invalid Email Format Accepted
- Invalid Email Domain Accepted
- Mobile Number Less Than 10 Digits Accepted
- Mobile Number More Than 10 Digits Accepted
- Mobile Number Accepts Alphanumeric Characters
- Mobile Number Accepts Special Characters

Medium Severity Defects

- Name Field Accepts Numeric Values

- Name Field Accepts Special Characters
- Name Field Accepts Alphanumeric Characters
- Address Field Accepts Numeric Values
- Address Field Accepts Special Characters

Low Severity Defects

- Address Field Accepts Insufficient Input
-

12. CHALLENGES FACED

- Jenkins Pipeline Configuration
 - GitHub Integration
 - Selenium Synchronization Issues
 - Dynamic Element Handling
 - Browser Compatibility
 - Test Dependency Management
-

13. BENEFITS OF AUTOMATION

- Faster Execution
 - Improved Accuracy
 - Reusability
 - Continuous Integration
 - Better Reporting
 - Reduced Manual Effort
-

14. CONCLUSION

The Automation Testing Practice Website Automation Framework successfully automates the major functionalities of the application using Selenium WebDriver, Java, TestNG, Maven, Jenkins, and GitHub.

The framework provides robust automation capabilities, detailed reporting through Extent Reports, continuous integration through Jenkins, and improved testing efficiency. Several validation-related defects were identified during testing, demonstrating the effectiveness of the automation framework in detecting application issues.

This project significantly improves software quality, reduces testing effort, and serves as a scalable automation solution for future enhancements.